Resolución — Taller SQL sobre HR 4 — JOIN y combinación de tablas

Bases de Datos I · resolución paso a paso

Antes de empezar

- Cada ejercicio muestra lo que se pide, la idea, los pasos, la consulta, su resultado en HR y un aviso de error común.
- Las consultas están escritas para Oracle Database 19c y solo leen el esquema HR. Los resultados salen de ejecutarlas ahí.
- Hay más de una forma correcta de resolver casi todos los ejercicios; cuando conviene, se muestra una variante.

Ejercicio 1 — Empleados con su departamento

Nivel: Básico · **Tablas:** `employees`, `departments`

Se pide. Listar los empleados con el nombre del departamento donde trabajan.

Mostrar: nombre, apellido y nombre del departamento, ordenados por departamento, apellido y nombre.

Idea. Un `JOIN` une dos tablas con una condición: aquí, la columna `department_id`, que las dos tienen.

Pasos.

1. `hr.employees` guarda el `department_id` de cada empleado y `hr.departments` guarda el nombre de ese departamento.
2. `JOIN hr.departments d ON d.department_id = e.department_id` empareja cada empleado con su departamento.
3. En el `SELECT` van las dos columnas del empleado y `d.department_name`.
4. Se ordena por departamento, apellido y nombre.

```
SELECT e.first_name,
       e.last_name,
       d.department_name
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
ORDER BY d.department_name, e.last_name, e.first_name;
```

Resultado en HR. 106 filas y 3 columnas; se muestran las primeras 6 filas:

```
FIRST_NAME  LAST_NAME  DEPARTMENT_NAME
----------  ---------  --------------
William     Gietz      Accounting
Shelley     Higgins    Accounting
Jennifer    Whalen     Administration
Lex         Garcia     Executive
Steven      King       Executive
Neena       Yang       Executive
```

Ojo. Un `JOIN` interno solo devuelve las filas que tienen pareja: Kimberely Grant no tiene departamento y no aparece, por eso salen 106 filas y no 107. Y `NATURAL JOIN` no sirve aquí: las dos tablas comparten `department_id` y también `manager_id`, así que une por las dos y devuelve solo 32 filas.

Variante. Cuando la columna de unión se llama igual en las dos tablas se puede usar `USING`. Con `USING` esa columna no lleva prefijo de alias en el resto de la consulta.

```
SELECT e.first_name,
       e.last_name,
       d.department_name
FROM hr.employees e
JOIN hr.departments d USING (department_id)
ORDER BY d.department_name, e.last_name, e.first_name;
```

Ejercicio 2 — Tres tablas en una consulta

Nivel: Básico · **Tablas:** `employees`, `departments`, `jobs`

Se pide. Mostrar los empleados del departamento Finance con el título de su cargo y el rango salarial permitido para ese cargo.

Mostrar: nombre, apellido, cargo (`job_title`), salario, salario mínimo y salario máximo del cargo, de mayor a menor salario y, si empatan, por apellido.

Idea. Cada `JOIN` agrega una tabla al resultado: las uniones se encadenan, y cada una lleva su propia condición `ON`.

Pasos.

1. Se parte de `hr.employees e` y se une `hr.departments d` para poder filtrar por el nombre del departamento.
2. Se une `hr.jobs j` por `job_id` para traer el título del cargo y sus salarios mínimo y máximo.
3. El filtro `d.department_name = 'Finance'` va en el `WHERE`.
4. Se ordena por salario descendente y, en empate, por apellido.

```
SELECT e.first_name,
       e.last_name,
       j.job_title,
       e.salary,
       j.min_salary,
       j.max_salary
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
JOIN hr.jobs j ON j.job_id = e.job_id
WHERE d.department_name = 'Finance'
ORDER BY e.salary DESC, e.last_name;
```

Resultado en HR. 6 filas y 6 columnas:

```
FIRST_NAME   LAST_NAME  JOB_TITLE        SALARY  MIN_SALARY  MAX_SALARY
-----------  ---------  ---------------  ------  ----------  ----------
Nancy        Gruenberg  Finance Manager   12008        8200       16000
Daniel       Faviet     Accountant         9000        4200        9000
John         Chen       Accountant         8200        4200        9000
Jose Manuel  Urman      Accountant         7800        4200        9000
Ismael       Sciarra    Accountant         7700        4200        9000
Luis         Popp       Accountant         6900        4200        9000
```

Ojo. Si una tabla se une sin condición, Oracle no avisa: devuelve el producto cartesiano, cada fila con todas las de la otra. Los 6 empleados de Finance combinados con los 19 cargos con `CROSS JOIN` darían 114 filas.

Ejercicio 3 — Todos los empleados, con o sin departamento

Nivel: Intermedio · **Tablas:** `employees`, `departments`

Se pide. Listar a todos los empleados con el nombre de su departamento. A quien no tenga departamento hay que mostrarlo con el texto «Sin departamento».

Mostrar: nombre, apellido y departamento, ordenados por departamento (primero quien no tiene), apellido y nombre.

Idea. `LEFT JOIN` conserva todas las filas de la tabla de la izquierda, tengan o no pareja. Donde no hay pareja, las columnas de la derecha salen en `NULL`.

Pasos.

1. La tabla que manda es `hr.employees`: todas sus filas tienen que salir, así que va a la izquierda.
2. `LEFT JOIN hr.departments d ON d.department_id = e.department_id` agrega el departamento; para quien no tiene, `d.department_name` sale `NULL`.
3. `NVL(d.department_name, 'Sin departamento')` cambia ese `NULL` por el texto pedido.
4. `ORDER BY d.department_name NULLS FIRST` pone primero a quien no tiene departamento.

```
SELECT e.first_name,
       e.last_name,
       NVL(d.department_name, 'Sin departamento') AS departamento
FROM hr.employees e
LEFT JOIN hr.departments d ON d.department_id = e.department_id
ORDER BY d.department_name NULLS FIRST, e.last_name, e.first_name;
```

Resultado en HR. 107 filas y 3 columnas; se muestran las primeras 6 filas:

```
FIRST_NAME  LAST_NAME  DEPARTAMENTO
----------  ---------  ---------------
Kimberely   Grant      Sin departamento
William     Gietz      Accounting
Shelley     Higgins    Accounting
Jennifer    Whalen     Administration
Lex         Garcia     Executive
Steven      King       Executive
```

Ojo. El `NULL` aparece siempre en las columnas de la tabla de la derecha. Aquí salen las 107 filas: la de Kimberely Grant se conserva y `NVL` pone el texto. Con el `JOIN` interno del ejercicio 1 esa persona desaparecía.

Variante. Escrito al revés da lo mismo: `RIGHT JOIN` conserva todas las filas de la tabla de la derecha. Casi siempre se prefiere `LEFT JOIN` con la tabla que manda a la izquierda, porque se lee de corrido.

```
SELECT e.first_name,
       e.last_name,
       NVL(d.department_name, 'Sin departamento') AS departamento
FROM hr.departments d
RIGHT JOIN hr.employees e ON d.department_id = e.department_id
ORDER BY d.department_name NULLS FIRST, e.last_name, e.first_name;
```

Ejercicio 4 — Departamentos que nadie ocupa

Nivel: Intermedio · **Tablas:** `departments`, `employees`

Se pide. Listar los departamentos que no tienen ningún empleado.

Mostrar: código y nombre del departamento, ordenados por código.

Idea. Un `LEFT JOIN` deja en `NULL` las columnas de la derecha cuando no hay pareja. Si se filtra por ese `NULL`, quedan solo las filas sin pareja.

Pasos.

1. Manda `hr.departments`: se buscan sus filas que no tienen coincidencia en `hr.employees`.
2. `LEFT JOIN hr.employees e ON e.department_id = d.department_id` empareja cada departamento con sus empleados.
3. Un departamento vacío queda con `e.employee_id` en `NULL`.
4. `WHERE e.employee_id IS NULL` deja solo a esos departamentos.

```
SELECT d.department_id,
       d.department_name
FROM hr.departments d
LEFT JOIN hr.employees e ON e.department_id = d.department_id
WHERE e.employee_id IS NULL
ORDER BY d.department_id;
```

Resultado en HR. 16 filas y 2 columnas; se muestran las primeras 6 filas:

```
DEPARTMENT_ID  DEPARTMENT_NAME
-------------  --------------------
          120  Treasury
          130  Corporate Tax
          140  Control And Credit
          150  Shareholder Services
          160  Benefits
          170  Manufacturing
```

Ojo. Se prueba la llave primaria de la tabla de la derecha (`employee_id`) porque en una fila real nunca es `NULL`: si lo es, es que no hubo pareja. En HR salen 16 departamentos, del 120 al 270; solo los once primeros (10 a 110) tienen empleados.

Variante. `NOT EXISTS` da el mismo resultado y se lee más cerca del enunciado:

```
SELECT d.department_id,
       d.department_name
FROM hr.departments d
WHERE NOT EXISTS (SELECT 1
                  FROM hr.employees e
                  WHERE e.department_id = d.department_id)
ORDER BY d.department_id;
```

Ejercicio 5 — El cruce completo con FULL JOIN

Nivel: Avanzado · **Tablas:** `employees`, `departments`

Se pide. Mostrar en una sola lista a los empleados que no tienen departamento y a los departamentos que no tienen empleados.

Mostrar: nombre y apellido del empleado (si lo hay) y nombre del departamento (si lo hay), con el empleado sin departamento primero y luego los departamentos por nombre.

Idea. `FULL OUTER JOIN` conserva las filas sin pareja de los dos lados. Después, un `WHERE` se queda solo con las que no tienen pareja.

Pasos.

1. `FULL OUTER JOIN` empareja empleados con departamentos y además conserva a los empleados sin departamento y a los departamentos sin empleados.
2. Un empleado sin departamento deja `d.department_id` en `NULL`; un departamento vacío deja `e.employee_id` en `NULL`.
3. `WHERE e.employee_id IS NULL OR d.department_id IS NULL` se queda con esas dos clases de fila.
4. `NULLS FIRST` sube al empleado sin departamento, y los departamentos se ordenan por nombre.

```
SELECT e.first_name,
       e.last_name,
       d.department_name
FROM hr.employees e
FULL OUTER JOIN hr.departments d ON d.department_id = e.department_id
WHERE e.employee_id IS NULL
   OR d.department_id IS NULL
ORDER BY d.department_name NULLS FIRST, e.last_name;
```

Resultado en HR. 17 filas y 3 columnas; se muestran las primeras 6 filas:

```
FIRST_NAME  LAST_NAME  DEPARTMENT_NAME
----------  ---------  ------------------
Kimberely   Grant      (null)
(null)      (null)     Benefits
(null)      (null)     Construction
(null)      (null)     Contracting
(null)      (null)     Control And Credit
(null)      (null)     Corporate Tax
```

Ojo. Es una consulta de auditoría: busca datos sin pareja en los dos lados. En HR salen 17 filas, un empleado sin departamento y 16 departamentos vacíos. `OUTER` es opcional: `FULL JOIN` y `FULL OUTER JOIN` son lo mismo.

Ejercicio 6 — Cada empleado con su jefe

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Mostrar a cada empleado con el nombre de su jefe directo. Quien no tenga jefe debe salir con el texto «Sin jefe».

Mostrar: empleado, jefe y salario del empleado, ordenados por jefe (primero quien no tiene), nombre del jefe, apellido y nombre del empleado.

Idea. Una tabla puede unirse consigo misma si se usan dos alias: `e` hace de empleado y `m` de jefe. La condición enlaza la llave del jefe (`employee_id`) con la columna `manager_id` del empleado.

Pasos.

1. `hr.employees e` es el empleado y `hr.employees m` es la misma tabla leída como jefe: dos alias, una tabla.
2. La condición es `m.employee_id = e.manager_id`: el jefe es la fila cuyo identificador está en `manager_id`.
3. Se usa `LEFT JOIN` porque Steven King no tiene `manager_id` y no debe desaparecer.
4. `NVL2(m.employee_id, nombre del jefe, 'Sin jefe')` devuelve el nombre si hay jefe y el texto si no lo hay.

```
SELECT e.first_name || ' ' || e.last_name AS empleado,
       NVL2(m.employee_id, m.first_name || ' ' || m.last_name, 'Sin jefe')
         AS jefe,
       e.salary
FROM hr.employees e
LEFT JOIN hr.employees m ON m.employee_id = e.manager_id
ORDER BY m.last_name NULLS FIRST, m.first_name, e.last_name, e.first_name;
```

Resultado en HR. 107 filas y 3 columnas; se muestran las primeras 6 filas:

```
EMPLEADO         JEFE              SALARY
---------------  ----------------  ------
Steven King      Sin jefe           24000
Elizabeth Bates  Gerald Cambrault    7300
Harrison Bloom   Gerald Cambrault   10000
Tayler Fox       Gerald Cambrault    9600
Sundita Kumar    Gerald Cambrault    6100
Lisa Ozer        Gerald Cambrault   11500
```

Ojo. `NVL(m.first_name || ' ' || m.last_name, 'Sin jefe')` no funciona: en Oracle, concatenar `NULL` con un espacio da un espacio y no `NULL`, así que Steven King saldría con el jefe en blanco. Por eso se pregunta por la llave del jefe con `NVL2`.

Ejercicio 7 — Contratados antes que su jefe

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Encontrar a los empleados que fueron contratados antes que su jefe directo.

Mostrar: empleado, su fecha de contratación, jefe y fecha de contratación del jefe (DD/MM/AAAA), ordenados por la fecha del empleado y, si empatan, por apellido.

Idea. En una autorreunión se pueden comparar las columnas de las dos lecturas: aquí, la fecha del empleado contra la de su jefe.

Pasos.

1. Se unen e (empleado) y `m` (jefe) con `m.employee_id = e.manager_id`.
2. Es un `JOIN` interno: Steven King no tiene jefe y no puede cumplir una comparación de fechas.
3. El filtro `e.hire_date < m.hire_date` compara las dos fechas.
4. Las fechas se muestran con `TO_CHAR` y se ordena por la fecha real del empleado.

```
SELECT e.first_name || ' ' || e.last_name AS empleado,
       TO_CHAR(e.hire_date, 'DD/MM/YYYY') AS fecha_empleado,
       m.first_name || ' ' || m.last_name AS jefe,
       TO_CHAR(m.hire_date, 'DD/MM/YYYY') AS fecha_jefe
FROM hr.employees e
JOIN hr.employees m ON m.employee_id = e.manager_id
WHERE e.hire_date < m.hire_date
ORDER BY e.hire_date, e.last_name, e.first_name;
```

Resultado en HR. 37 filas y 4 columnas; se muestran las primeras 6 filas:

```
EMPLEADO         FECHA_EMPLEADO  JEFE             FECHA_JEFE
---------------  --------------  ---------------  ----------
Lex Garcia       13/01/2011      Steven King      17/06/2013
Hermann Brown    07/06/2012      Neena Yang       21/09/2015
Shelley Higgins  07/06/2012      Neena Yang       21/09/2015
Susan Jacobs     07/06/2012      Neena Yang       21/09/2015
Daniel Faviet    16/08/2012      Nancy Gruenberg  17/08/2012
Nancy Gruenberg  17/08/2012      Neena Yang       21/09/2015
```

Ojo. Son 37 de los 106 empleados que tienen jefe: mucha gente entró antes que quien hoy la dirige. Con otra comparación, `e.salary > m.salary`, la misma consulta devuelve a quienes ganan más que su jefe, y son solo dos: Ellen Abel y Lisa Ozer, ambas representantes de ventas que ganan 500 más que su gerente.

Ejercicio 8 — El jefe del jefe

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Para cada empleado del departamento IT (`department_id` 60), mostrar su jefe directo y el jefe de su jefe.

Mostrar: empleado, jefe directo y jefe del jefe, ordenados por apellido y nombre del empleado.

Idea. Cada nivel que se sube en la jerarquía es una autorreunión más, con un alias nuevo.

Pasos.

1. e es el empleado, `m1` su jefe directo (`m1.employee_id = e.manager_id`) y `m2` el jefe de ese jefe (`m2.employee_id = m1.manager_id`).
2. Son tres lecturas de la misma tabla, cada una con su alias.
3. El filtro `e.department_id = 60` se aplica al empleado.
4. Se ordena por apellido y nombre.

```
SELECT e.first_name || ' ' || e.last_name AS empleado,
       m1.first_name || ' ' || m1.last_name AS jefe,
       m2.first_name || ' ' || m2.last_name AS jefe_del_jefe
FROM hr.employees e
JOIN hr.employees m1 ON m1.employee_id = e.manager_id
JOIN hr.employees m2 ON m2.employee_id = m1.manager_id
WHERE e.department_id = 60
ORDER BY e.last_name, e.first_name;
```

Resultado en HR. 5 filas y 3 columnas:

```
EMPLEADO         JEFE             JEFE_DEL_JEFE
---------------  ---------------  -------------
Valli Jackson    Alexander James  Lex Garcia
Alexander James  Lex Garcia       Steven King
Bruce Miller     Alexander James  Lex Garcia
Diana Nguyen     Alexander James  Lex Garcia
David Williams   Alexander James  Lex Garcia
```

Ojo. Con `JOIN` interno, quien no tenga jefe del jefe quedaría fuera. Para subir un número cualquiera de niveles no se encadenan alias: se usan las consultas jerárquicas con `CONNECT BY`.

Variante. Con `LEFT JOIN` en los dos pasos también se conservan los empleados cuya cadena de jefes es más corta. Aquí el resultado es el mismo porque los cinco tienen jefe del jefe.

```
SELECT e.first_name || ' ' || e.last_name AS empleado,
       m1.first_name || ' ' || m1.last_name AS jefe,
       m2.first_name || ' ' || m2.last_name AS jefe_del_jefe
FROM hr.employees e
LEFT JOIN hr.employees m1 ON m1.employee_id = e.manager_id
LEFT JOIN hr.employees m2 ON m2.employee_id = m1.manager_id
WHERE e.department_id = 60
ORDER BY e.last_name, e.first_name;
```

Ejercicio 9 — Quién gana más que quién

Nivel: Avanzado · **Tablas:** `employees`

Se pide. En el departamento IT (`department_id` 60), listar todas las parejas de empleados en las que el primero gana más que el segundo.

Mostrar: quien gana más y su salario, quien gana menos y su salario, de mayor a menor diferencia de salario y, si empatan, por apellidos.

Idea. No todos los `JOIN` comparan con =. Una condición con <, > o `BETWEEN` también une: cada fila se empareja con todas las que cumplen la desigualdad.

Pasos.

1. a y b son dos lecturas de `hr.employees`.
2. `b.department_id = a.department_id` pone a las dos personas en el mismo departamento.
3. `b.salary < a.salary` empareja a cada empleado con todos los que ganan menos que él.
4. El filtro `a.department_id = 60` limita el cálculo a IT, y se ordena por la diferencia de salario.

```
SELECT a.first_name || ' ' || a.last_name AS gana_mas,
       a.salary AS salario_a,
       b.first_name || ' ' || b.last_name AS gana_menos,
       b.salary AS salario_b
FROM hr.employees a
JOIN hr.employees b ON b.department_id = a.department_id
                   AND b.salary < a.salary
WHERE a.department_id = 60
ORDER BY a.salary - b.salary DESC, a.last_name, b.last_name;
```

Resultado en HR. 9 filas y 4 columnas; se muestran las primeras 6 filas:

```
GANA_MAS         SALARIO_A  GANA_MENOS      SALARIO_B
---------------  ---------  --------------  ---------
Alexander James       9000  Diana Nguyen         4200
Alexander James       9000  Valli Jackson        4800
Alexander James       9000  David Williams       4800
Alexander James       9000  Bruce Miller         6000
Bruce Miller          6000  Diana Nguyen         4200
Bruce Miller          6000  Valli Jackson        4800
```

Ojo. La desigualdad es estricta: David Williams y Valli Jackson ganan lo mismo (4800) y no forman pareja. Y como solo se acepta un sentido (<), cada pareja sale una vez y no dos. Con cinco empleados en IT salen 9 parejas.

Ejercicio 10 — Cuántos empleados tiene cada departamento, incluso los vacíos

Nivel: Avanzado · **Tablas:** `departments`, `employees`

Se pide. Mostrar todos los departamentos con la cantidad de empleados que tiene cada uno, incluidos los que tienen cero.

Mostrar: nombre del departamento y cantidad de empleados, de menor a mayor cantidad y, si empatan, por nombre del departamento.

Idea. `COUNT(columna)` no cuenta los `NULL`. Sobre un `LEFT JOIN`, los departamentos vacíos dan 0 y no desaparecen.

Pasos.

1. Manda `hr.departments`, que va a la izquierda de un `LEFT JOIN` con `hr.employees`.
2. Se agrupa por departamento.
3. `COUNT(e.employee_id)` cuenta solo las filas donde hubo empleado.
4. Se ordena de menor a mayor para ver primero los vacíos.

```
SELECT d.department_name,
       COUNT(e.employee_id) AS empleados
FROM hr.departments d
LEFT JOIN hr.employees e ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name
ORDER BY empleados, d.department_name;
```

Resultado en HR. 27 filas y 2 columnas; se muestran las primeras 6 filas:

```
DEPARTMENT_NAME     EMPLEADOS
------------------  ---------
Benefits                    0
Construction                0
Contracting                 0
Control And Credit          0
Corporate Tax               0
Government Sales            0
```

Ojo. Con `COUNT(*)` los departamentos vacíos saldrían con 1: el `LEFT JOIN` produce una fila (con `NULL`) por cada uno y `COUNT(*)` cuenta filas. Para contar solo lo que existe se cuenta una columna de la tabla de la derecha.

Variante. Una subconsulta correlacionada en el `SELECT` cuenta sin `JOIN` ni `GROUP BY` y da las mismas 27 filas:

```
SELECT d.department_name,
       (SELECT COUNT(*)
        FROM hr.employees e
        WHERE e.department_id = d.department_id) AS empleados
FROM hr.departments d
ORDER BY empleados, d.department_name;
```

Ejercicio 11 — Cinco tablas: de empleado a región

Nivel: Avanzado · **Tablas:** `employees`, `departments`, `locations`, `countries`, `regions`

Se pide. Mostrar los empleados que trabajan fuera de Estados Unidos, con su departamento, la ciudad, el código del país y la región.

Mostrar: nombre, apellido, departamento, ciudad, código del país y región, ordenados por nombre del país, ciudad, apellido y nombre.

Idea. Cada `JOIN` da un paso en el camino de los datos: empleado, departamento, ubicación, país y región.

Pasos.

1. `hr.employees` se une con `hr.departments` por `department_id`.
2. `hr.departments` se une con `hr.locations` por `location_id`.
3. `hr.locations` se une con `hr.countries` por `country_id`.
4. `hr.countries` se une con `hr.regions` por `region_id`.
5. El filtro `c.country_id <> 'US'` va en el `WHERE`, y se ordena por país, ciudad y empleado.

```
SELECT e.first_name,
       e.last_name,
       d.department_name,
       l.city,
       c.country_id,
       r.region_name
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
JOIN hr.locations l ON l.location_id = d.location_id
JOIN hr.countries c ON c.country_id = l.country_id
JOIN hr.regions r ON r.region_id = c.region_id
WHERE c.country_id <> 'US'
ORDER BY c.country_name, l.city, e.last_name, e.first_name;
```

Resultado en HR. 38 filas y 6 columnas; se muestran las primeras 6 filas:

```
FIRST_NAME  LAST_NAME  DEPARTMENT_NAME   CITY     COUNTRY_ID  REGION_NAME
----------  ---------  ----------------  -------  ----------  -----------
Pat         Davis      Marketing         Toronto  CA          Americas
Michael     Martinez   Marketing         Toronto  CA          Americas
Hermann     Brown      Public Relations  Munich   DE          Europe
Susan       Jacobs     Human Resources   London   GB          Europe
Ellen       Abel       Sales             Oxford   GB          Europe
Sundar      Ande       Sales             Oxford   GB          Europe
```

Ojo. Con tantas tablas conviene un alias corto por tabla y un `JOIN` por línea. Quien no tiene departamento queda fuera desde el primer `JOIN`, igual que en el ejercicio 1. Salen 38 empleados: 35 en el Reino Unido, 2 en Canadá y 1 en Alemania.

Ejercicio 12 — Países con ubicaciones y ubicaciones con departamentos

Nivel: Reto · **Tablas:** `countries`, `locations`, `departments`

Se pide. Para cada país de la tabla, mostrar cuántas ubicaciones tiene y cuántas de ellas tienen al menos un departamento. Los países sin ubicaciones también deben salir.

Mostrar: nombre del país, cantidad de ubicaciones y cantidad de ubicaciones con departamento, de mayor a menor cantidad de ubicaciones y, si empatan, por nombre del país.

Idea. Dos `LEFT JOIN` encadenados: cada uno conserva lo que no encuentra pareja en el paso siguiente. Como una ubicación puede tener varios departamentos, se cuenta con `DISTINCT`.

Pasos.

1. Manda `hr.countries`: todos los países tienen que salir.
2. `LEFT JOIN hr.locations l` trae las ubicaciones de cada país; un país sin ubicaciones deja `l` en `NULL`.
3. `LEFT JOIN hr.departments d` trae los departamentos de cada ubicación; si `l.location_id` es `NULL`, tampoco hay departamentos.
4. `COUNT(DISTINCT l.location_id)` cuenta ubicaciones y `COUNT(DISTINCT d.location_id)` cuenta las que tienen algún departamento.

```
SELECT c.country_name,
       COUNT(DISTINCT l.location_id) AS ubicaciones,
       COUNT(DISTINCT d.location_id) AS con_departamentos
FROM hr.countries c
LEFT JOIN hr.locations l ON l.country_id = c.country_id
LEFT JOIN hr.departments d ON d.location_id = l.location_id
GROUP BY c.country_id, c.country_name
ORDER BY ubicaciones DESC, c.country_name;
```

Resultado en HR. 25 filas y 3 columnas; se muestran las primeras 6 filas:

```
COUNTRY_NAME                   UBICACIONES  CON_DEPARTAMENTOS
-----------------------------  -----------  -----------------
United States of America                 4                  3
United Kingdom of Great Bri...           3                  2
Canada                                   2                  1
Italy                                    2                  0
Japan                                    2                  0
Switzerland                              2                  0
```

Ojo. Sin `DISTINCT`, Estados Unidos saldría con 24 ubicaciones y 23 con departamento, en lugar de 4 y 3: Seattle tiene 21 departamentos y el `JOIN` repite la ubicación una vez por cada uno. En HR hay 11 países sin ubicaciones, que salen con 0 y 0.